Python 3 Master Cheat Sheet

Data Structures, OOP, Functional Patterns, AsyncIO & Memory Management

1. Data Types & Built-in Data Structures

- **Primitives:** `int`, `float`, `str`, `bool`, `bytes`, `None`.
- **Built-in Collections:** `list` (ordered, mutable), `tuple` (ordered, immutable), `dict` (key-value hash map, preserves insertion order), `set` (unique elements).

```
# List Comprehension & Dict Comprehension
squares = [x**2 for x in range(10) if x % 2 == 0]
char_map = {char: idx for idx, char in enumerate("PYTHON")}
```

2. Functions, Decorators & Generators

- **Decorators:** Functions taking another function to extend behavior without modifying source code.
- **Generators:** Use `yield` to produce lazy sequence evaluators with minimal memory footprint.

```
# Decorator Pattern
def log_execution(func):
    def wrapper(*args, **kwargs):
        print(f"Executing {func.__name__}")
        return func(*args, **kwargs)
    return wrapper

# Generator Example
def fibonacci(limit):
    a, b = 0, 1
    for _ in range(limit):
        yield a
        a, b = b, a + b
```

3. OOP & Magic (Dunder) Methods

```
class Vector:
    def __init__(self, x: float, y: float):
        self.x = x
        self.y = y

    def __add__(self, other: "Vector") -> "Vector":
        return Vector(self.x + other.x, self.y + other.y)

    def __repr__(self) -> str:
        return f"Vector({self.x}, {self.y})"

v1 = Vector(2, 4)
v2 = Vector(1, 3)
print(v1 + v2)  # Output: Vector(3, 7)
```

4. Asynchronous Python (AsyncIO) & Context Managers

```
import asyncio

# Custom Context Manager
class ManagedFile:
    def __init__(self, filename): self.filename = filename
    def __enter__(self):
        self.file = open(self.filename, 'w')
        return self.file
    def __exit__(self, exc_type, exc_val, exc_tb):
        if self.file: self.file.close()

# Async Coroutines
async def fetch_data(id: int):
    await asyncio.sleep(1)
    return {"id": id, "data": "payload"}

async def main():
    results = await asyncio.gather(fetch_data(1), fetch_data(2))
    print(results)
```